

Поиск подстроки в строке

Постановка задачи поиска

- Заданы две строки s и x . Длина первой строки n , длина второй – m ($0 \leq m \leq n$)
- Требуется определить, является ли строка x подстрокой строки s (найти первое вхождение x в s)

Прямой (линейный) поиск

- Сводится к последовательным сравнениям отдельных символов

Прямой (линейный) поиск

```
found = False
```

```
i = 0
```

```
while not found and (i<=n-m):
```

```
    j = 0
```

```
    while (j<m) and (s[i+j]==x[j]): j++
```

```
    if j==m: found = True
```

```
    else: i++
```

```
if found: вернуть i
```

```
else: вернуть -1
```

Прямой (линейный) поиск

- Этот алгоритм требует достаточно больших временных затрат, поскольку, когда n значительно больше m , количество выполняемых сравнений равно $(n-m)*m \approx n*m$
- Написать без использования логической переменной

Прямой (линейный) поиск

```
i = 0
```

```
while (i <= n-m):
```

```
    j = 0
```

```
    while (j < m) and (s[i+j] == x[j]): j++
```

```
    if j == m: вернуть i
```

```
    i++
```

```
вернуть -1
```


Алгоритм Бойера–Мура

- Разработан Робертом Бойером и Джеем Муром в 1977 году

Алгоритм Бойера–Мура

- Поиск ведется от начала строки s , но с конца искомой подстроки x , для которой формируется таблица, размерность которой равна количеству всех возможных символов. Элементами таблицы являются расстояния от последнего символа искомой подстроки x до ее каждого символа (если в x встречаются одинаковые символы, то в таблицу заносится расстояние до ближайшего из них). Если символ в x не входит, то в соответствующую ячейку таблицы заносится m – длина подстроки x

Алгоритм Бойера–Мура

- Когда очередной символ подстроки не совпадает с очередным символом строки s , для последнего текущего символа строки из таблицы расстояний определяется соответствующее расстояние, после чего x сдвигается вправо на соответствующее число позиций

Алгоритм Бойера–Мура

- При анализе производительности алгоритма, его авторы показали, что почти всегда, кроме специально построенных примеров, алгоритм требует значительно меньше n сравнений
- В самых благоприятных случаях число сравнений равно n/m (когда последний символ искомой строки x всегда попадает на несовпадающий символ строки s)

Алгоритм Бойера–Мура

- Пример:

И	Н	Ф	О	Р	М	А	Т	И	К	А
2	9	8	7	6	5	4	3	2	1	4

- Пример:
- МИЛА_МАЛО_МЫЛАСЬ_МЫЛОМ
- МЫЛО

Алгоритм Бойера–Мура

n = длина строки s

m = длина подстроки x

d – массив сдвигов, исходно всем элементам присвоить значение m

for $i = 0$ to $m-2$:

$d[\text{ord}(x[i])] = m - i - 1$ # ord – код символа

Алгоритм Бойера–Мура

$i = 0$

found = False

while ($i < n - m + 1$) and not found:

$j = m - 1$

 while ($j > 0$) and ($s[i+j] == x[j]$): $j--$

 if $j == 0$: found = True

 else $i += d[\text{ord}(s[i+m])]$

if found: вывод $i+1$

else: вывод -1

Алгоритм Кнута–Морриса–Пратта

- Алгоритм был разработан Дональдом Эрвином Кнутом и Воном Рональдом Праттом и, независимо от них, Джеймсом Моррисом. Результаты своей работы они опубликовали совместно в 1977 году
- Алгоритм фактически требует только N сравнений даже в самом плохом случае

Алгоритм Кнута–Морриса–Пратта

- Пусть началом строки (префиксом) называется ее часть, получающаяся в результате отбрасывания нескольких букв с ее конца. Тогда при переходе из одного состояния в другое сохраняется информация о том, какое максимальное начало строки x является концом прочитанной части строки s , а номер состояния и есть длина этого начала, то есть значение соответствующего элемента массива d
- Пусть j – номер символа, входящего в искомую подстроку x , $d[j]$ есть длина максимальной последовательности символов s , предшествующих позиции j , которая полностью совпадает с началом x

Алгоритм Кнута–Морриса–Пратта

n = длина строки s , m = длина подстроки x

$st = x + \# + s$

$d[0] = 0$

$i = 0$

while ($d[i] \neq m$) and ($i < n+m$):

$i++$

$len = d[i-1]$

 while ($st[i] \neq st[len+1]$) and ($len > 0$):

$len = d[len-1]$

 if ($st[len+1] == st[i]$): $d[i] = len+1$

 else: $d[i] = 0$

if $d[i] == m$: вывод ($i-2*m$)

else: вывод -1

Алгоритм Карпа–Рабина

- это алгоритм поиска строки, который ищет шаблон, то есть подстроку, в тексте, используя хеширование. Он был разработан в 1987 году Майклом Рабином и Ричардом Карпом

Алгоритм Карпа–Рабина

```
function RabinKarp(s, x) {  
    hx = hash(x) // сумма кодов символов  
    hs = hash(s[0..m-1])  
    for i from 0 to (n-m+1):  
        if hs == hx и x посимвольно совпадает с m  
           символами строки начиная с i-го:  
            return i  
        hs = hash(s[i+1..i+m]) // удаляя код первого  
                               символа и добавляя код последнего  
    return -1
```